



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES
MULTIPLATAFORMA
Departamento de Informática

PROYECTO



Eventvs Mêrida

Definición de proyecto, resumen y justificación

Autor/es: David Muñoz Collado, Adrián Pérez Morales, Eva Retamar Muñoz
Curso Académico: 2025/2026
Tutor Proyecto : María Francisca Roncero Holgado

Índice

1. Resumen ejecutivo y alcance del proyecto	3
2. Justificación	4
2.1. Beneficios esperados	4
2.2. Integración con la industria extremeña	7
2.3. Análisis de productos similares	8
2.4. Participación en ODS	10
3. Metodología, temporalización y organización del equipo	12
3.1. Metodología utilizada	13
3.2. Organización del repositorio y trabajo con ramas	14
3.3. Organización del equipo	15
3.4. Temporalización del proyecto	16
3.5. Comunicación y coordinación	18
3.6. Herramientas utilizadas para la organización	19
4. Stack tecnológico	20
4.1. Frontend móvil	20
4.2. Backend	22
4.3. Base de datos	23
4.4. Almacenamiento de imágenes	24
4.5. Panel web de administración	25
4.6. Landing page	26
4.7. Web de restablecimiento de contraseña	26
4.8. Despliegue	27
4.9. Relación con decisiones de arquitectura	28
5. Mejoras futuras y escalabilidad de la solución	29
5.1. Mejoras futuras	29
5.2. Escalabilidad de la solución	33
5.3. Resumen	35
6. Conclusiones	36

1. Resumen ejecutivo y alcance del proyecto

Eventvs Mérida es una plataforma digital orientada a centralizar la información sobre eventos culturales, sociales, deportivos y de ocio que tienen lugar en la ciudad de Mérida. El proyecto surge como respuesta a un problema detectado: la información sobre actividades y eventos suele encontrarse repartida en diferentes páginas web, redes sociales, agendas municipales o medios de comunicación, lo que dificulta que ciudadanos y turistas puedan conocer de forma rápida qué actividades hay disponibles.

El objetivo principal del proyecto es ofrecer una herramienta sencilla y accesible que permita consultar eventos desde una única aplicación, facilitando la búsqueda, el filtrado por categorías, la consulta de detalles, la visualización de ubicaciones en el mapa y el guardado de eventos de interés. De esta forma, Eventvs Mérida pretende mejorar la difusión de la oferta cultural y de ocio de la ciudad, fomentar la participación ciudadana y apoyar a organizadores, entidades y establecimientos que realizan actividades.

Aunque la idea inicial del proyecto se centraba en una aplicación móvil básica para consultar eventos, durante el desarrollo el alcance se ha ampliado. Actualmente, el sistema está formado por varios componentes conectados entre sí: una aplicación móvil desarrollada en Flutter, un backend/API REST desarrollado en Spring Boot, una base de datos PostgreSQL alojada en Supabase, almacenamiento de imágenes en Supabase Storage, una landing page, un panel web de administración, una página de restablecimiento de contraseña y scripts auxiliares para la carga de eventos.

La aplicación móvil permite acceder a funcionalidades públicas sin necesidad de iniciar sesión, como consultar eventos, buscar, filtrar y ver ubicaciones en el mapa. Además, incorpora registro e inicio de sesión para usuarios, permitiendo guardar eventos favoritos, consultar eventos guardados y editar el perfil personal.

El sistema también diferencia distintos roles de usuario. Los usuarios registrados pueden utilizar las funcionalidades personalizadas de la aplicación. Los organizadores pueden crear, actualizar y eliminar sus propios eventos. Los administradores, además, disponen de acceso al panel web de administración, desde el que pueden gestionar usuarios, eventos, organizadores, categorías y roles.

El backend centraliza la lógica de negocio y actúa como intermediario entre las interfaces de usuario y la base de datos. A través de la API REST se gestionan operaciones como autenticación, registro, consulta de eventos, filtrado, favoritos, administración, subida de imágenes y recuperación de contraseña. La base de datos se

encuentra alojada en Supabase y las imágenes se almacenan en Supabase Storage, evitando guardar archivos pesados directamente en PostgreSQL.

También se han desarrollado scripts auxiliares en Python para descargar eventos desde fuentes externas, limpiar y normalizar los datos, obtener coordenadas y subir eventos a la API. Estos scripts permiten facilitar la carga inicial o periódica de información en el sistema.

El proyecto se centra inicialmente en la ciudad de Mérida, pero su estructura permite que pueda escalar en el futuro a otras localidades extremeñas. La arquitectura utilizada permite añadir nuevas ciudades, nuevas categorías de eventos, más fuentes de datos, funcionalidades de notificación, recomendaciones personalizadas o integración con sistemas externos.

En resumen, Eventvs Mérida ha evolucionado desde una idea inicial de catálogo de eventos hasta una solución más completa formada por aplicación móvil, backend, base de datos, almacenamiento, administración web, recuperación de contraseña, scripts de carga y despliegue en servicios cloud.

2. Justificación

2.1. Beneficios esperados

Beneficios para los usuarios

La aplicación permite a los usuarios consultar de forma rápida y sencilla los eventos disponibles en Mérida. Gracias al listado de eventos, la búsqueda, el filtrado por categorías y la visualización en mapa, el usuario puede encontrar actividades de interés sin tener que revisar diferentes fuentes de información.

Además, los usuarios registrados pueden guardar eventos favoritos y consultarlos posteriormente desde su perfil. Esto mejora la experiencia de uso y permite que cada usuario organice mejor las actividades a las que quiere asistir.

También se facilita el acceso a la información básica de cada evento, como título, descripción, fecha, localización, imagen y categoría. De esta forma, la aplicación actúa como una agenda digital centralizada.

Beneficios para organizadores

El proyecto también beneficia a organizadores, asociaciones, establecimientos o entidades que realizan eventos en Mérida. Los usuarios con rol de organizador pueden crear, editar y eliminar sus propios eventos desde la aplicación, lo que permite mantener la información actualizada y aumentar la visibilidad de sus actividades.

Esta funcionalidad ayuda a que pequeños organizadores o entidades locales puedan publicar sus eventos sin depender únicamente de redes sociales, carteles físicos o medios externos.

Beneficios para administradores

El sistema cuenta con un panel web de administración que permite gestionar usuarios, eventos, organizadores, categorías y roles. Esto facilita el control global del sistema y permite mantener la información organizada.

El panel de administración también incluye un dashboard con contadores y gráficos, lo que permite tener una visión general del estado de la plataforma.

Beneficios técnicos

Desde el punto de vista técnico, el proyecto permite aplicar una arquitectura completa formada por frontend móvil, backend/API REST, base de datos, almacenamiento de imágenes, panel web, landing page y despliegue en servicios cloud.

El backend desarrollado en Spring Boot centraliza la lógica de negocio y se comunica con una base de datos PostgreSQL alojada en Supabase. Las imágenes se almacenan en Supabase Storage, evitando guardar archivos pesados directamente en la base de datos. Además, el backend se encuentra desplegado en Render y las webs en Vercel, lo que permite probar el sistema desde cualquier dispositivo con conexión a Internet.

También se han desarrollado scripts auxiliares en Python para la carga de eventos desde fuentes externas, lo que mejora la capacidad de mantenimiento y actualización del catálogo de eventos.

Beneficios educativos

El desarrollo del proyecto ha permitido poner en práctica y reforzar conocimientos adquiridos durante el ciclo formativo, integrando diferentes áreas del desarrollo de software. Además, durante el proceso también hemos aprendido nuevas tecnologías, herramientas y formas de trabajo que no se habían tratado en profundidad durante el curso, pero que eran necesarias para poder completar el proyecto.

Entre los conocimientos aplicados y ampliados se encuentran:

- Desarrollo móvil con Flutter y Dart.
- Desarrollo backend con Spring Boot.
- Creación y consumo de APIs REST.
- Uso de bases de datos PostgreSQL mediante Supabase.
- Gestión de imágenes con almacenamiento externo mediante Supabase Storage.
- Seguridad, autenticación, sesiones, roles y permisos.
- Despliegue de servicios en plataformas cloud como Render y Vercel.
- Trabajo con Git y GitHub para control de versiones y trabajo colaborativo.
- Desarrollo web con HTML, CSS, JavaScript, Bootstrap y Angular.
- Automatización mediante scripts de Python.
- Uso de servicios externos para geolocalización y mapas.
- Gestión de variables de entorno y configuración de servicios desplegados.
- Documentación técnica y documentación orientada al usuario final.
- Resolución de errores reales derivados de la integración entre frontend, backend, base de datos y servicios externos.

Además, al tratarse de un proyecto en equipo, ha sido necesario organizar tareas, coordinar cambios, resolver conflictos, repartir responsabilidades y documentar el trabajo realizado. Esto nos ha permitido acercarnos a una forma de trabajo más parecida a la que se utiliza en un entorno profesional.

2.2. Integración con la industria extremeña

Eventvs Mêrida se integra dentro del contexto cultural, turístico y tecnológico de Extremadura. La ciudad de Mêrida tiene un gran peso patrimonial y cultural, con una amplia oferta de actividades, eventos, teatro, música, deporte, talleres y propuestas de ocio.

El proyecto se relaciona principalmente con los siguientes sectores:

Sector cultural y turístico

La aplicación contribuye a mejorar la difusión de actividades culturales y turísticas de Mêrida. Al centralizar los eventos en una única plataforma, se facilita que ciudadanos y turistas puedan descubrir actividades disponibles en la ciudad.

Esto puede beneficiar a teatros, museos, asociaciones culturales, establecimientos, salas de conciertos, entidades municipales y empresas relacionadas con el turismo.

Además, Mêrida cuenta con un importante patrimonio histórico y artístico, por lo que una herramienta de este tipo puede servir como apoyo para promocionar eventos vinculados a espacios culturales y monumentales.

Sector tecnológico

El proyecto utiliza tecnologías actuales para resolver un problema real de difusión cultural. La solución combina desarrollo móvil, backend, bases de datos, servicios cloud, almacenamiento de imágenes, geolocalización y paneles de administración.

Esto encaja con la digitalización de servicios y con la creación de herramientas tecnológicas aplicadas al entorno local.

Sector educativo

El proyecto también tiene relación con el ámbito educativo, ya que ha sido desarrollado dentro del contexto de un ciclo formativo. Permite aplicar conocimientos técnicos en un caso práctico real y acercar la formación profesional al desarrollo de soluciones útiles para el entorno.

Posibles casos de uso en Extremadura

La estructura del proyecto permitiría ampliar la aplicación a otros municipios extremeños en el futuro. Aunque inicialmente está centrada en Mérida, podría adaptarse para incluir eventos de otras localidades como Cáceres, Badajoz, Trujillo, Plasencia u otros municipios con actividad cultural y turística.

También podría utilizarse como herramienta de apoyo para oficinas de turismo, asociaciones culturales, ayuntamientos o entidades que necesiten difundir eventos de forma centralizada.

2.3. Análisis de productos similares

Para justificar la utilidad del proyecto, se han analizado varias plataformas existentes que ofrecen información sobre eventos o actividades culturales. Aunque estas plataformas cumplen una función similar, Eventvs Mérida busca aportar una experiencia más centrada en la ciudad, con funcionalidades móviles, mapa, favoritos, roles de organizador y administración propia.

Agenda de eventos del Ayuntamiento de Mérida - [Link](#)



La agenda del Ayuntamiento de Mérida es una fuente oficial de información sobre actividades culturales, sociales y deportivas de la ciudad.

Entre sus puntos fuertes destaca que se trata de una fuente institucional y fiable. Además, ofrece información relevante sobre eventos locales.

Sin embargo, presenta algunas limitaciones desde el punto de vista de experiencia de usuario, ya que no está planteada como una aplicación móvil personalizada. Tampoco ofrece funcionalidades como guardar eventos favoritos, recibir una experiencia adaptada al usuario o gestionar eventos desde perfiles de organizador.

Portal de Turismo de Extremadura - [Link](#)



El portal oficial de Turismo de Extremadura ofrece información turística y cultural de toda la región. Su principal ventaja es que tiene una cobertura regional y permite conocer actividades de diferentes municipios.

Como limitación, al abarcar toda Extremadura, la información específica de Mérida puede quedar menos destacada. Además, no se centra exclusivamente en una experiencia móvil de eventos ni incorpora funcionalidades personalizadas como favoritos o gestión de eventos por organizadores.

ViralAgenda - [Link](#)



ViralAgenda es una plataforma privada que recoge eventos culturales y actividades de diferentes zonas. Su ventaja principal es que centraliza información de distintas fuentes y permite consultar eventos sin registro.

Como limitación, la información puede depender de aportaciones externas y no siempre tiene el mismo nivel de detalle o control. Además, no está enfocada específicamente a Mérida ni ofrece un sistema propio de administración, roles, favoritos y gestión de eventos desde una app móvil.

Plataforma	Cobertura	Fuente	Personalización	Gestión de eventos	Enfoque principal
Agenda Ayuntamiento de Mérida	Local	Oficial	Baja	No disponible para organizadores externos	Agenda institucional
Turismo de Extremadura	Regional	Oficial	Baja	No orientada a organizadores locales	Información turística regional
ViralAgenda	Local/regional	Privada	Baja	Limitada	Agenda cultural general
Eventvs Mérida	Local, ampliable	Plataforma propia	Favoritos, perfil y roles	Sí, mediante organizadores y administración	App móvil centralizada de eventos

Eventvs Mérida se diferencia al ofrecer una solución más integrada, con aplicación móvil, backend propio, panel de administración, usuarios con roles, gestión de eventos, favoritos, mapas y posibilidad de ampliación futura.

2.4. Participación en ODS

El proyecto también se relaciona con varios Objetivos de Desarrollo Sostenible, ya que contribuye a mejorar el acceso a la cultura, fomentar la participación ciudadana, apoyar la economía local y promover el uso de soluciones digitales.

ODS 3: Salud y bienestar



La aplicación puede ayudar a difundir eventos deportivos, actividades al aire libre, talleres de bienestar, rutas, actividades sociales y propuestas que favorezcan la participación y el ocio saludable.

Al facilitar el acceso a este tipo de actividades, Eventvs Mêrida puede contribuir a que más personas conozcan opciones para mantenerse activas, relacionarse con otras personas y participar en eventos que mejoren su bienestar físico y social.

ODS 4: Educación de calidad



La aplicación también permite difundir talleres, charlas, conferencias, cursos y actividades culturales o formativas que se celebren en Mêrida.

De esta forma, los usuarios pueden descubrir oportunidades de aprendizaje no formal y actividades de enriquecimiento cultural que de otro modo podrían pasar desapercibidas.

ODS 8: Trabajo decente y crecimiento económico



Eventvs Mêrida puede favorecer la visibilidad de organizadores, comercios, asociaciones y entidades locales que realizan actividades. Al mejorar la difusión de eventos, se puede aumentar la asistencia y apoyar indirectamente la actividad económica local.

Esto puede beneficiar a establecimientos, salas, espacios culturales, guías, empresas turísticas y pequeños organizadores.

ODS 9: Industria, innovación e infraestructura



El proyecto aplica tecnología para resolver un problema real de información y difusión cultural. Utiliza una infraestructura digital formada por aplicación móvil, backend, base de datos, almacenamiento cloud y panel web.

Esto contribuye a la digitalización de servicios locales y demuestra cómo una solución tecnológica puede mejorar la organización y el acceso a información de interés ciudadano.

ODS 11: Ciudades y comunidades sostenibles



La aplicación ayuda a mejorar el acceso a la oferta cultural y de ocio de la ciudad, fomentando una comunidad más participativa e informada.

Además, al centralizar la información en formato digital, se reduce la dependencia de cartelería física o información dispersa, contribuyendo a una comunicación más eficiente y sostenible.

3. Metodología, temporalización y organización del equipo

Para el desarrollo de Eventvs Mêrida hemos seguido una metodología de trabajo iterativa e incremental. Aunque el proyecto se ha desarrollado dentro de un entorno académico, hemos intentado organizar el trabajo de una forma similar a un proyecto real, dividiendo el desarrollo en fases, repartiendo tareas entre los miembros del equipo y utilizando herramientas de control de versiones y seguimiento del trabajo.

El proyecto ha ido evolucionando desde una idea inicial centrada en una aplicación móvil de consulta de eventos hasta una solución más completa, formada por aplicación

móvil, backend, base de datos, almacenamiento de imágenes, panel de administración, landing page, página de recuperación de contraseña y scripts auxiliares.

Durante el desarrollo hemos trabajado por funcionalidades, priorizando primero las partes principales de la aplicación y añadiendo después mejoras, validaciones, administración, despliegue y documentación.

3.1. Metodología utilizada

La metodología utilizada se ha basado en un desarrollo ágil e incremental, organizando el trabajo en pequeñas tareas y revisando continuamente el estado del proyecto.

En lugar de desarrollar todo el proyecto de una vez, se ha trabajado por bloques funcionales. Primero se definió la idea general, después se creó una primera versión funcional y finalmente se fueron añadiendo mejoras, despliegues, validaciones y documentación.

Las principales fases de trabajo han sido:

- Análisis inicial de la idea.
- Definición de funcionalidades principales.
- Diseño de la arquitectura general.
- Desarrollo del backend y la base de datos.
- Desarrollo del frontend móvil.
- Integración entre frontend y backend.
- Despliegue.
- Desarrollo del panel web de administración.
- Implementación de funcionalidades complementarias.
- Pruebas funcionales.
- Documentación final.

Para organizar el trabajo se han utilizado herramientas como GitHub, ramas de trabajo y, en caso de seguimiento visual de tareas, herramientas tipo Trello.

El uso de GitHub ha permitido mantener un control de versiones del proyecto, trabajar en ramas separadas y coordinar los cambios entre los distintos miembros del equipo.

3.2. Organización del repositorio y trabajo con ramas

El proyecto se ha gestionado mediante un repositorio en GitHub, donde se ha almacenado el código fuente de los distintos componentes: aplicación móvil, backend, panel de administración, landing page, web de restablecimiento de contraseña, scripts y documentación.

Para organizar el trabajo con Git, cada miembro del equipo trabajaba en su propia rama. En estas ramas individuales se desarrollaban nuevas funcionalidades, correcciones o mejoras. Una vez terminados los cambios, se subían al repositorio y se integraban en la rama común de desarrollo, llamada develop.

La rama develop se utilizaba como punto de integración del trabajo del equipo. En ella se comprobaba que los cambios funcionaban correctamente junto con el resto del proyecto y que no generaban errores importantes.

Cuando el contenido de develop estaba revisado, probado y estable, se integraba en la rama main, que se mantenía como versión principal y estable del proyecto.

El flujo de trabajo seguido ha sido:

1. Cada miembro trabajaba en su propia rama.
2. Se desarrollaban funcionalidades, correcciones o mejoras.
3. Los cambios se subían al repositorio.
4. Se integraban en la rama común `develop`.
5. Se probaba que los cambios funcionaban correctamente.
6. Cuando `develop` estaba estable, se integraba en `main`.

Este sistema nos ha permitido trabajar de forma más organizada y evitar subir directamente a la rama principal cambios sin probar. Además, ha facilitado el trabajo en paralelo, la revisión de errores y la integración progresiva de funcionalidades.

3.3. Organización del equipo

La organización del trabajo se ha realizado a partir de una lista de tareas ordenadas según la prioridad y la dependencia entre funcionalidades. A medida que avanzaba el proyecto, estas tareas se iban asignando entre los miembros del equipo según la disponibilidad, la dificultad y el estado del desarrollo.

En general, todos los miembros del equipo hemos participado en diferentes partes del proyecto, ya que muchas funcionalidades requerían coordinación entre frontend, backend, base de datos y despliegue. Por este motivo, aunque algunas tareas estuvieran asignadas principalmente a una persona, ha sido habitual revisar, probar o corregir partes desarrolladas por otros compañeros.

El trabajo se ha repartido principalmente en las siguientes áreas:

- Desarrollo de la aplicación móvil en Flutter.
- Desarrollo del backend/API REST en Spring Boot.
- Diseño y mantenimiento de la base de datos en Supabase.
- Desarrollo del panel web de administración.
- Desarrollo de la landing page y la web de restablecimiento de contraseña.
- Creación y uso de scripts auxiliares para la carga de eventos.
- Pruebas, corrección de errores y documentación.

Aunque todos hemos colaborado en el conjunto del proyecto, el desarrollo del backend y del panel de administración ha tenido mayor participación por parte de dos compañeros del equipo, mientras que el desarrollo de funcionalidades en la aplicación móvil, integración con la API, pruebas y documentación también se ha trabajado de forma conjunta.

Esta forma de organización nos ha permitido avanzar de manera flexible, adaptándonos a los problemas que iban apareciendo durante el desarrollo. Además, al trabajar sobre funcionalidades conectadas entre sí, ha sido necesario mantener una comunicación constante para comprobar que los datos enviados desde la aplicación móvil coincidían con lo que esperaba el backend y que los cambios realizados no afectaban al resto del sistema.

3.4. Temporalización del proyecto

El desarrollo del proyecto se ha dividido en tres grandes fases: fase inicial, segunda fase y fase final.

Fase inicial: descripción y análisis de la solución

En esta primera fase se trabajó la definición de la idea, el problema que se quería resolver y el alcance inicial del proyecto.

Durante esta fase se realizaron tareas como:

- Análisis del problema de dispersión de eventos en Mérida.
- Definición de los objetivos del proyecto.
- Identificación de usuarios y roles.
- Definición inicial de funcionalidades.
- Análisis de productos similares.
- Primer planteamiento de arquitectura.
- Definición inicial de tecnologías.
- Creación del repositorio y organización inicial del proyecto.
- Elaboración del prototipo inicial en Figma.

En esta fase, la solución se planteó principalmente como una aplicación móvil. La idea inicial era que la consulta de eventos y la gestión por parte de usuarios organizadores y administradores se realizara desde la propia app.

Segunda fase: implementación del producto mínimo viable

En la segunda fase se comenzó el desarrollo de las funcionalidades principales del producto mínimo viable. El objetivo fue conseguir una primera versión funcional de la aplicación, conectando frontend, backend, base de datos y almacenamiento externo.

Durante esta fase se trabajó en:

- Creación del backend con Spring Boot.
- Definición de entidades principales: Usuario, Evento, Categoría y Rol.

- Creación de DTOs, mappers, controladores, servicios y repositorios.
- Conexión con la base de datos PostgreSQL en Supabase.
- Integración con Supabase Storage para la subida de imágenes.
- Desarrollo de scripts de carga y normalización de eventos.
- Desarrollo de la aplicación móvil en Flutter.
- Implementación de pantallas principales.
- Listado y detalle de eventos.
- Registro e inicio de sesión.
- Primera implementación del mapa, centrado inicialmente en la localización de Mérida.
- Gestión de eventos guardados.

En esta fase se mantuvo el enfoque inicial de que la aplicación móvil fuera el centro principal del sistema. El mapa se incorporó en una primera versión para mostrar la localización base de Mérida, dejando para fases posteriores la gestión más precisa de coordenadas por evento.

Fase final: desarrollo, integración y despliegue

En la fase final se completaron funcionalidades, se corrigieron errores, se mejoró la integración entre componentes y se realizó el despliegue de los distintos servicios.

Durante esta fase también se amplió el alcance inicial del proyecto. Aunque al principio se había planteado que la gestión administrativa se realizará desde la app móvil, finalmente se decidió desarrollar un panel web de administración para facilitar la gestión de usuarios, eventos, organizadores, categorías y roles desde una interfaz más cómoda para los administradores.

Durante esta fase se trabajó en:

- Gestión avanzada de roles: usuario registrado, organizador y administrador.
- Ajustes en la autenticación y sesiones.
- Creación, edición y eliminación de eventos por organizadores.

- Mejora la ubicación de eventos mediante la latitud y la longitud.
- Selección y búsqueda de ubicación en mapa al crear o editar eventos.
- Panel web de administración.
- Landing page del proyecto.
- Página de restablecimiento de contraseña.
- Despliegue de las webs en Vercel.
- Incluir información sobre la utilización de la aplicación.
- Utilizar paginación de eventos para mejorar el rendimiento de la aplicación.
- Pruebas funcionales.
- Corrección de errores.
- Documentación final del proyecto.

Esta fase permitió transformar la idea inicial en una solución más completa, incorporando no solo la aplicación móvil, sino también herramientas de administración, recuperación de contraseña, gestión más precisa de ubicaciones, scripts auxiliares y despliegue en servicios cloud.

3.5. Comunicación y coordinación

Al tratarse de un proyecto en equipo, la comunicación ha sido fundamental para coordinar el desarrollo. Durante el proyecto se han realizado revisiones del estado de las tareas, resolución conjunta de errores y coordinación para integrar las distintas partes del sistema.

Uno de los retos principales ha sido la integración entre frontend, backend, base de datos y servicios externos. Por ello, ha sido necesario mantener una comunicación continua para asegurar que los datos enviados desde Flutter coincidían con los DTOs del backend, que los endpoints funcionaban correctamente y que las funcionalidades desplegadas estaban actualizadas.

También ha sido necesario coordinar el uso de ramas en GitHub para evitar conflictos y asegurar que los cambios se integran correctamente.

3.6. Herramientas utilizadas para la organización

Durante el desarrollo se han utilizado distintas herramientas de apoyo:

Herramienta	Uso
GitHub	Control de versiones y repositorio del proyecto
Git	Gestión de ramas, commits y cambios
Trello / tablero de tareas	Organización y seguimiento de tareas
Discord / WhatsApp / reuniones	Comunicación del equipo
Android Studio	Desarrollo y pruebas de la app Flutter
IntelliJ IDEA	Desarrollo del backend Spring Boot
Visual Studio Code	Desarrollo web, scripts y edición de archivos
Render	Despliegue del backend
Vercel	Despliegue de landing page, panel web y recuperación de contraseña
Supabase	Base de datos y almacenamiento de imágenes

4. Stack tecnológico

Para el desarrollo de Eventvs Mérida se ha utilizado un stack tecnológico dividido en varios bloques: aplicación móvil, backend, base de datos, almacenamiento, webs auxiliares, scripts, despliegue y herramientas de organización. La elección de estas tecnologías se realizó teniendo en cuenta las necesidades del proyecto, los conocimientos adquiridos durante el ciclo y la posibilidad de crear una solución completa, escalable y mantenible.

4.1. Frontend móvil

La aplicación móvil se ha desarrollado con Flutter y Dart.

Flutter nos ha permitido crear una aplicación móvil moderna y con una interfaz visual adaptable. Además, al ser una tecnología multiplataforma, permite que el proyecto pueda ampliarse en un futuro a otros entornos, como iOS o web, aunque en esta versión el objetivo principal sea Android.

En la aplicación móvil se han utilizado diferentes paquetes para cubrir funcionalidades concretas:

Tecnología / paquete	Uso en el proyecto
Flutter	Desarrollo de la aplicación móvil
Dart	Lenguaje de programación utilizado en Flutter
go_router	Gestión de rutas y navegación entre pantallas
http	Comunicación con la API REST del backend

shared_preferences	Persistencia de datos sencillos, como usuario o preferencias
flutter_secure_storage	Almacenamiento seguro de información sensible
flutter_map	Visualización de mapas
latlong2	Gestión de coordenadas de latitud y longitud
table_calendar	Calendario de eventos
intl	Formateo de fechas e internacionalización
image_picker	Selección de imágenes desde el dispositivo
url_launcher	Apertura de enlaces externos
share_plus	Compartición de eventos
tutorial_coach_mark	Tutorial guiado inicial

4.2. Backend

El backend se ha desarrollado con Spring Boot, utilizando Java como lenguaje principal.

Spring Boot se ha elegido porque permite crear una API REST robusta y organizada, separando el proyecto por capas: controladores, servicios, repositorios, entidades, DTOs y mappers. Esta estructura facilita el mantenimiento, la ampliación de funcionalidades y la localización de errores.

En el backend se han utilizado principalmente:

Tecnología	Uso en el proyecto
Java	Lenguaje principal del backend
Spring Boot	Framework principal del backend
Spring Web	Creación de controladores y endpoints REST
Spring Data JPA	Acceso a base de datos mediante repositorios
Spring Security	Autenticación, sesiones, roles y permisos
Maven	Gestión de dependencias y construcción del proyecto
Lombok	Reducción de código repetitivo en entidades y servicios

BCryptPasswordEncoder	Cifrado de contraseñas
springdoc-openapi	Documentación de endpoints
RestClient	Comunicación con servicios externos como Supabase Storage

4.3. Base de datos

La base de datos utilizada es PostgreSQL, alojada en Supabase.

Supabase ha permitido disponer de una base de datos en la nube sin necesidad de mantener un servidor propio. Además, PostgreSQL es una base de datos relacional robusta y adecuada para el modelo de datos del proyecto, donde existen relaciones entre usuarios, roles, eventos, categorías y eventos guardados.

Tecnología	Uso en el proyecto
PostgreSQL	Base de datos relacional
Supabase	Plataforma cloud donde se aloja la base de datos
JPA/Hibernate	Mapeo entre entidades Java y tablas de base de datos

La base de datos almacena la información principal del sistema:

- Usuarios
- Roles
- Eventos
- Categorías
- Eventos guardados por usuario
- Tokens de recuperación de contraseña
- Tokens relacionados con autenticación

4.4. Almacenamiento de imágenes

Para almacenar imágenes se ha utilizado Supabase Storage.

No se guardan las imágenes directamente en la base de datos, ya que esto haría que la base de datos creciera innecesariamente y complicaría la gestión de archivos. En su lugar, las imágenes se almacenan en buckets de Supabase Storage y en la base de datos se guarda la URL o el path correspondiente.

Bucket	Uso
imagenesEvento	Imágenes asociadas a eventos
imagenesPerfil	Fotos de perfil de usuarios

4.5. Panel web de administración

El panel web de administración se ha desarrollado con HTML, CSS, JavaScript, Bootstrap y la plantilla SB Admin.

Este panel permite a los administradores gestionar usuarios, eventos, organizadores, categorías y roles desde una interfaz web. También incluye un dashboard con contadores y gráficos para visualizar información general del sistema.

Tecnología	Uso en el proyecto
HTML	Estructura de las páginas
CSS	Estilos personalizados
JavaScript	Lógica del panel y consumo de la API
Bootstrap	Diseño responsive y componentes visuales
SB Admin	Plantilla base del panel de administración
Simple Datatables	Tablas dinámicas
Font Awesome	Iconos
Chart.js / gráficos	Visualización de datos

El panel consume la misma API REST que la aplicación móvil, por lo que comparte backend, base de datos y lógica de negocio.

4.6. Landing page

La landing page del proyecto se ha desarrollado con Angular 19.

Su objetivo es presentar el proyecto, mostrar información general, permitir la descarga de la APK y facilitar el acceso al repositorio de GitHub.

Tecnología	Uso en el proyecto
Angular 19	Desarrollo de la landing page
TypeScript	Lenguaje utilizado en Angular
SCSS	Estilos de la landing
i18n	Traducciones en varios idiomas
Vercel	Despliegue de la landing page

La landing page incluye diseño responsive, animaciones, selector de idioma y secciones informativas sobre la aplicación y el equipo.

4.7. Web de restablecimiento de contraseña

La web de restablecimiento de contraseña se ha desarrollado con HTML, CSS y JavaScript.

Esta página permite que un usuario introduzca una nueva contraseña mediante un token recibido por URL. El token se envía al backend, donde se valida si es correcto, si ha caducado o si ya ha sido utilizado.

Tecnología	Uso en el proyecto
HTML	Estructura de la página
CSS	Diseño visual
JavaScript	Validación y envío de datos al backend
Bootstrap	Componentes visuales
SweetAlert2	Mensajes de éxito y error
i18n	Traducciones
Vercel	Despliegue de la página

4.8. Despliegue

Para el despliegue del proyecto se han utilizado diferentes plataformas cloud según el tipo de componente.

Componente	Plataforma de despliegue
Backend/API REST	Render

Landing page	Vercel
Panel web de administración	Vercel
Web de restablecimiento de contraseña	Vercel
Base de datos	Supabase
Almacenamiento de imágenes	Supabase Storage

El backend se ha desplegado en Render mediante Docker, mientras que las webs se han desplegado en Vercel. Supabase se utiliza como servicio cloud para base de datos y almacenamiento.

Esta organización permite que la aplicación pueda probarse desde un dispositivo móvil sin necesidad de ejecutar manualmente todos los servicios en local.

4.9. Relación con decisiones de arquitectura

Las principales decisiones tecnológicas del proyecto se documentaron mediante ADR. Estas decisiones justifican la elección de las tecnologías principales:

ADR-001: Selección de Flutter para el desarrollo del frontend móvil.

ADR-002: Selección de Spring Boot para el backend.

ADR-003: Selección de Supabase para la base de datos y autenticación.

ADR-004: Selección de Render para el despliegue del backend.

Estas decisiones han servido para mantener una línea tecnológica coherente durante el desarrollo y justificar por qué se eligieron determinadas herramientas frente a otras alternativas.

5. Mejoras futuras y escalabilidad de la solución

Aunque Eventvs Mérida cuenta con una versión funcional que permite consultar eventos, buscar, filtrar, guardar favoritos, gestionar usuarios y administrar eventos, existen varias mejoras que podrían implementarse en el futuro para ampliar el alcance del proyecto y mejorar la experiencia de uso.

Además, la arquitectura utilizada permite que la aplicación pueda escalar progresivamente, incorporando nuevas funcionalidades, nuevos perfiles de usuario, más municipios o nuevas fuentes de información sin tener que rehacer el sistema completo.

5.1. Mejoras futuras

Sistema de notificaciones

Una de las mejoras principales sería incorporar un sistema completo de notificaciones. Esto permitiría avisar a los usuarios sobre nuevos eventos, cambios en eventos guardados, eventos próximos o actividades relacionadas con sus categorías favoritas.

Las notificaciones podrían ser:

- Notificaciones push en la aplicación móvil.
- Notificaciones por correo electrónico.
- Avisos sobre eventos guardados próximos.
- Avisos sobre nuevas actividades según intereses del usuario.

Esta funcionalidad mejoraría la participación de los usuarios y haría que la aplicación fuera más útil en el día a día.

Recomendaciones personalizadas

Otra mejora futura sería añadir un sistema de recomendaciones personalizadas. Actualmente, el usuario puede buscar y filtrar eventos, pero en futuras versiones la aplicación podría sugerir eventos según sus intereses, categorías favoritas, eventos guardados o historial de uso.

Por ejemplo, si un usuario guarda muchos eventos de teatro, la aplicación podría destacar nuevos eventos relacionados con esa categoría.

Ampliación a otras ciudades

Aunque el proyecto está centrado inicialmente en Mérida, su estructura permitirá ampliarlo a otras localidades de Extremadura.

Para ello se podría añadir un campo de ciudad o municipio asociado a cada evento, permitiendo filtrar eventos por localidad. De esta forma, Eventvs Mêrida podría evolucionar hacia una plataforma regional de eventos culturales y de ocio.

Algunas posibles ciudades o zonas de ampliación serían:

- Badajoz
- Cáceres
- Trujillo
- Plasencia
- Almendralejo
- Zafra
- Otras localidades extremeñas
- Mancomunidades

Validación previa de eventos creados por organizadores

Actualmente, los usuarios organizadores pueden gestionar sus propios eventos. Como mejora futura, se podría añadir un sistema de revisión previa, donde los eventos creados por organizadores quedarán pendientes de validación hasta que un administrador los apruebe.

Esto permitiría mejorar el control de calidad de la información publicada y evitar contenido incorrecto, duplicado o no adecuado.

El flujo podría ser:

1. El organizador crea un evento.
2. El evento queda en estado pendiente.
3. El administrador revisa el evento desde el panel web.

4. El administrador aprueba o rechaza el evento.
5. Si se aprueba, el evento aparece públicamente en la aplicación.

Normalización de localizaciones

Una mejora futura sería incorporar un sistema de localizaciones normalizadas. Actualmente, al crear o editar un evento, el organizador puede escribir manualmente la localización del evento y seleccionar un punto en el mapa mediante latitud y longitud. Aunque esto permite guardar la ubicación, el texto de la localización puede quedar escrito de formas diferentes dependiendo de cada usuario.

Por ejemplo, un mismo lugar podría aparecer como:

Teatro Romano

Teatro romano de Mérida

C/ José Ramón Mérida, s/n

Para mejorar esto, se podría crear una tabla específica de localizaciones o espacios, donde se almacenaran lugares frecuentes de Mérida con un formato estándar.

De esta forma, al crear un evento, el organizador podría buscar y seleccionar una localización ya existente en lugar de escribirla manualmente. Si el lugar no existiera, podría añadirse como nueva localización, pero manteniendo un formato controlado.

Esta mejora permitiría:

- Evitar nombres duplicados o escritos de forma distinta.
- Mejorar la calidad de los datos.
- Mostrar direcciones más profesionales y coherentes.
- Facilitar búsquedas y filtros por ubicación.
- Reutilizar coordenadas sin tener que buscarlas cada vez.
- Reducir errores entre el texto de localización y el punto del mapa.

Con esta mejora, la gestión de ubicaciones sería más uniforme y la aplicación tendría una estructura de datos más profesional y escalable.

Compra o reserva de entradas

Otra posible mejora sería integrar un sistema de compra o reserva de entradas. Actualmente la aplicación muestra información sobre los eventos, pero en futuras versiones podría permitir al usuario acceder directamente a la compra o reserva desde la app.

Esta integración podría realizarse mediante:

- Enlaces externos a plataformas de venta.
- Integración con sistemas de ticketing.
- Reserva interna para eventos gratuitos o con aforo limitado.

Mejora del panel de administración

El panel web de administración podría ampliarse con más opciones de control y análisis.

Algunas mejoras posibles serían:

- Estadísticas más detalladas de eventos.
- Visualización de eventos más guardados.
- Control de actividad de organizadores.
- Gestión de eventos pendientes de aprobación.
- Filtros avanzados en tablas.
- Exportación de datos.
- Mejoras en gráficos y dashboard.

Esto permitiría que el administrador tuviera una visión más completa del uso de la plataforma.

Internacionalización completa

La landing page y la web de recuperación de contraseña ya incluyen soporte multi idioma. Como mejora futura, se podría ampliar la internacionalización a toda la aplicación móvil y al panel de administración.

Esto sería especialmente útil para turistas, ya que Mérida recibe visitantes nacionales e internacionales.

5.2. Escalabilidad de la solución

La solución se ha desarrollado con una arquitectura separada por componentes, lo que facilita su crecimiento y mantenimiento.

El sistema está formado por:

- Frontend móvil en Flutter.
- Backend/API REST en Spring Boot.
- Base de datos PostgreSQL en Supabase.
- Almacenamiento de imágenes en Supabase Storage.
- Panel web de administración.
- Landing page.
- Web de restablecimiento de contraseña.
- Scripts auxiliares.

Esta separación permite modificar o ampliar una parte del sistema sin afectar necesariamente al resto.

Escalabilidad funcional

La aplicación puede crecer añadiendo nuevas funcionalidades sin modificar por completo la estructura existente.

Por ejemplo, se podrían añadir nuevos módulos como:

- Notificaciones.
- Recomendaciones.
- Compra de entradas.
- Valoraciones de eventos.
- Comentarios.

- Agenda personalizada.
- Validación de eventos por administradores.

El backend ya está organizado por capas, con controladores, servicios, repositorios, entidades, DTOs y mappers. Esto facilita añadir nuevas funcionalidades de forma ordenada.

Escalabilidad geográfica

Aunque el proyecto se centra en Mérida, la estructura puede adaptarse para trabajar con más ciudades.

Para ello sería necesario ampliar el modelo de datos, añadiendo información relacionada con la ciudad o municipio del evento. A partir de ahí, la aplicación podría filtrar eventos por localidad y convertirse en una plataforma regional.

Esto permitiría que el proyecto creciera desde una aplicación local hacia una aplicación de eventos de Extremadura.

Escalabilidad técnica

El uso de servicios cloud facilita que el sistema pueda crecer sin depender de infraestructura local.

Actualmente se utilizan:

- Render para el backend.
- Vercel para las webs.
- Supabase para base de datos y almacenamiento.

Estos servicios permiten desplegar y mantener el proyecto online, facilitando futuras ampliaciones.

Además, el uso de una API REST permite que en el futuro puedan añadirse nuevos clientes, como:

- Aplicación iOS.
- Aplicación web para usuarios.
- Integraciones con terceros.

- Nuevas herramientas internas de administración.

Escalabilidad de datos

La base de datos PostgreSQL permite mantener un modelo relacional consistente, con entidades como usuarios, eventos, roles, categorías y eventos guardados.

En caso de crecer el volumen de datos, se podrían aplicar mejoras como:

- Paginación en más endpoints.
- Índices en campos de búsqueda.
- Filtros por ciudad, categoría o fecha.
- Optimización de consultas.
- Limpieza o archivado de eventos antiguos.

Estas mejoras permitirían mantener un buen rendimiento aunque aumente el número de eventos y usuarios.

Escalabilidad en almacenamiento de imágenes

Al utilizar Supabase Storage, las imágenes se almacenan fuera de la base de datos. Esto facilita que el sistema pueda gestionar más imágenes de eventos y usuarios sin sobrecargar PostgreSQL.

En el futuro se podrían añadir mejoras como:

- Compresión automática de imágenes.
- Generación de miniaturas.
- Organización de imágenes por carpetas o ciudades.
- Eliminación automática de imágenes no utilizadas.

5.3. Resumen

En conclusión, Eventvs Mérida es una solución que puede seguir creciendo tanto a nivel funcional como técnico. La separación entre aplicación móvil, backend, base de datos, almacenamiento, panel web y servicios desplegados permite que el proyecto sea mantenible y ampliable.

Las principales líneas de mejora se centran en incorporar notificaciones, recomendaciones personalizadas, validación de eventos, ampliación a otras ciudades, mejora del panel de administración, compra o reserva de entradas e integración con nuevas fuentes de datos.

Gracias a la arquitectura utilizada, el proyecto puede evolucionar desde una aplicación centrada en Mérida hacia una plataforma más amplia para la gestión y difusión de eventos en Extremadura.

6. Conclusiones

El desarrollo de Eventvs Mêrida nos ha permitido trabajar en un proyecto completo, desde la definición inicial de una idea hasta la creación de una solución funcional con aplicación móvil, backend, base de datos, almacenamiento de imágenes, panel de administración, landing page, recuperación de contraseña, scripts auxiliares y despliegue en servicios cloud.

Al inicio del proyecto partíamos de una idea más sencilla: crear una aplicación móvil para centralizar eventos de Mérida y facilitar su consulta. Sin embargo, a medida que avanzó el desarrollo, el proyecto fue creciendo y se incorporaron nuevas funcionalidades y componentes que hicieron que la solución fuera mucho más completa.

Uno de los aprendizajes más importantes ha sido entender la importancia de una buena organización entre frontend, backend y base de datos. Muchas funcionalidades no dependen solo de una pantalla o de un endpoint, sino de que todas las partes estén bien conectadas. Por ejemplo, para crear un evento no basta con diseñar un formulario en Flutter; también es necesario que el backend reciba correctamente los datos, que el DTO coincida con lo enviado desde la app, que la imagen se suba al almacenamiento, que el usuario tenga permisos y que finalmente el evento se guarde correctamente en la base de datos.

También hemos aprendido a resolver problemas reales de integración. Durante el desarrollo aparecieron errores relacionados con el envío de imágenes mediante multipart, la gestión de sesiones y permisos, el despliegue del backend, la conexión con Supabase, la localización de eventos en el mapa y la coordinación de cambios mediante Git. Estos problemas nos han obligado a investigar, probar soluciones y entender mejor cómo funciona una aplicación completa.

A nivel técnico, el proyecto nos ha permitido reforzar conocimientos adquiridos durante el ciclo y aprender otros nuevos. Hemos trabajado con Flutter, Spring Boot, APIs REST, PostgreSQL, Supabase, Supabase Storage, Render, Vercel, GitHub, JavaScript, Angular, Bootstrap y scripts en Python. Esta combinación de tecnologías nos ha dado una visión más amplia del desarrollo de software y de cómo se conectan las distintas partes de un sistema real.

El trabajo en equipo también ha sido una parte fundamental del proyecto. Hemos tenido que repartir tareas, coordinarnos, revisar cambios, resolver conflictos, integrar funcionalidades y tomar decisiones sobre el alcance del proyecto. Aunque cada miembro ha tenido más peso en determinadas áreas, todos hemos tenido que colaborar para que las distintas partes funcionaran correctamente entre sí.

Como mejora, si volviéramos a empezar el proyecto desde cero, dedicaríamos más tiempo al inicio a definir con mayor precisión la arquitectura, los endpoints, los DTOs y el flujo de datos entre frontend y backend. También habría sido útil cerrar antes algunas decisiones importantes, como la separación entre administración móvil y panel web, la gestión exacta de roles o el tratamiento de ubicaciones con latitud y longitud.

Otra mejora sería planificar desde el principio una estrategia de pruebas más clara. Aunque se han realizado pruebas funcionales durante el desarrollo, contar antes con un plan de pruebas más estructurado habría ayudado a detectar errores de forma más rápida y ordenada.

En conclusión, Eventvs Mêrida ha sido un proyecto muy completo que nos ha permitido aplicar conocimientos técnicos, aprender nuevas herramientas y enfrentarnos a problemas reales de desarrollo. El resultado es una plataforma funcional que centraliza eventos de Mêrida y que, además, tiene margen para seguir creciendo en el futuro, tanto en funcionalidades como en alcance geográfico.